StackOS

SMART CONTRACT AUDIT

ZOKYO.

Apr 22, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges

TECHNICAL SUMMARY

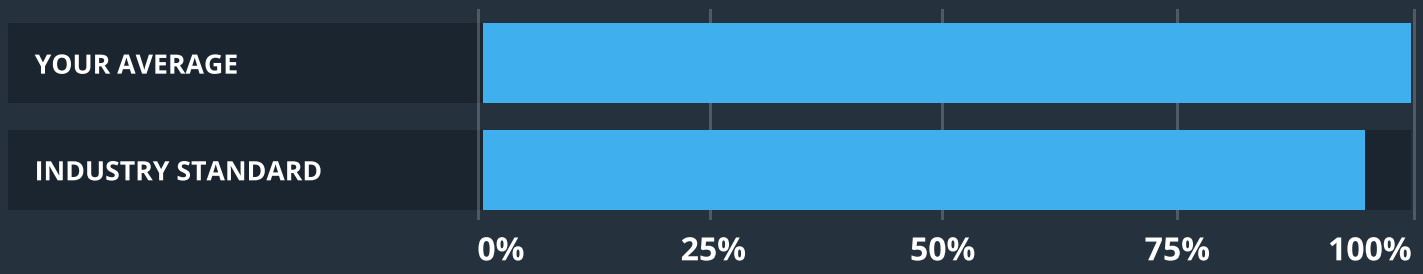
This document outlines the overall security of the StackOS smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the StackOS smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the StackOS team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied . . . 3
Executive Summary . . . 4
Structure and Organization of Document . . . 5
Manual Review . . . 6
Code Coverage and Test Results for all files . . . 9
Tests written by Zokyo Secured team . . . 9

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Unmarshal repository – https://gitlab.com/integro_io/idal/stackos-token-vesting-contract.
Last commit – db481d0d9e7b522d24250abf6de70e2959019596.

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of StackOS smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

EXECUTIVE SUMMARY

There were **neither critical nor major bugs** found in the contract during the audit. The overall contract's structure is well-formed and transparent. However, there is a list of recommended changes to be applied to the contract to increase its stability and overall code readability.

Despite the fact, the expected logic is managing all vestings by the owner, it should be careful with parameters to avoid mistakes during the vesting process.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

MANUAL REVIEW

The unexpected revert

MEDIUM | RESOLVED

If the tgeTimestamp (the constructor variable) is set to the far future, the calls to the withdrawFromVesting() and getWithdrawableAmount() will fail with the error: "SafeMath: Subtraction overflow".

Recommendation:
In calculateElapsedPeriods() function return zero in case of overflow and in withdrawFromVesting() function implement a proper require statement.

The SafeMath omission

LOW | RESOLVED

```
function withdrawFromVesting() external {
    Vesting storage vesting = getVesting(msg.sender);
    uint256 amountToPay = _getWithdrawableAmount(vesting);
    vesting.paidAmount += amountToPay;
    token.transfer(vesting.beneficiary, amountToPay);
    emit VestingWithdraw(msg.sender, amountToPay,
        vesting.vestingSchedule);
}
```

The withdrawFromVesting() function omits SafeMath addition.

Recommendation:

```
vesting.paidAmount = vesting.paidAmount.add(amountToPay);
```

Unnecessary multiplication

LOW | RESOLVED

```
cliffInPeriods: SECONDS_IN_MONTH.mul(1).div(SECONDS_IN_HOUR)
```

This line can be found throughout the code with the “useless” multiplication by 1.

Recommendation:
Remove .mul(1).

Parameter variable

INFORMATIONAL

```
function setTokenContract(address _tokenAddress) public onlyOwner {
token = IERC20(_tokenAddress);
}
```

The parameter variable can be IERC20 to avoid unnecessary “cast”.

Names of internal and private functions

INFORMATIONAL | RESOLVED

There are some internal and private functions the names of which should be prefixed with the underscore.

Name of the event

INFORMATIONAL | RESOLVED

```
event VestingWithdraw(
address beneficiary,
uint256 amount,
VestingSchedule vestingSchedule
);
```

The event name should be <noun><verb in the past> or <verb in the past>.

Recommendation:
Rename the event to VestingWithdrawn.

Other recommendations

- The workability of the whole contract is tightly coupled with the errorless behavior of the owner. To protect the users a bit, the contract may introduce a special counter of "allocated vestings" so as to ensure that there never be a situation of creating more vestings than there are tokens for.
- According to the code, the contract owner is allowed to "override" an already existing vesting which might decrease trust among the community. The recommendation is that the contract might check whether the vesting is already created and restrict the owner from the manipulation.

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Secured team

As part of our work assisting StackOS in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the StackOS contract requirements for details about issuance amounts and how the system handles these.

```
Contract: Vesting Test Cases
  ✓ shouldn't create vesting with zero amount (647ms)
  ✓ shouldn't set token address to the zero address (259ms)
  ✓ shouldn't add beneficiary if already exist (278ms)
  ✓ shouldn't add beneficiary if amount exceeds vesting schedule allocation (365ms)
  ✓ shouldn't create vesting if length of beneficiary not equal to vesting schedule or token
    allocation (397ms)
  ✓ shouldn't withdraw from vesting if TGE didn't start (3424ms)
  ✓ shouldn't withdraw tokens if amount is zero (395ms)
  ✓ should return withdrawable amount of tokens according to distribution type before
    period correct (1272ms)
  ✓ should return withdrawable amount of tokens after duration time correct (2066ms)
  ✓ should withdraw tokens according to distribution type before period correct (3536ms)
  ✓ should withdraw team token allocation after cliff period correct (629ms)
  ✓ should withdraw advisors token allocation after cliff period correct (688ms)
  ✓ should withdraw ecosystem token allocation after cliff period correct (901ms)
  ✓ should withdraw marketing token allocation after cliff period correct (693ms)
  ✓ should withdraw reserve token allocation after cliff period correct (776ms)
  ✓ should withdraw seedsale token allocation after cliff period correct (643ms)
  ✓ should withdraw strategicsale token allocation after cliff period correct (672ms)
  ✓ should withdraw privatesale token allocation after cliff period correct (790ms)
  ✓ should withdraw publicsale token allocation after cliff period correct (755ms)

19 passing (1m)
```

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	100.00	100.00	100.00	
VestingOS.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**100.00**	**100.00**	**100.00**	

We are grateful to have been given the opportunity to work with the StackOS team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the StackOS team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.